

	Deps Actions Status passing	coverage 100%																												
1 8 	2 7 PostHTML	3 9	4 5 twitter badge discontinued																											
PostHTML is a tool for transforming HTML/XML with JS plugins. PostHTML itself is very small. It includes only a HTML parser, a HTML node tree API and a node tree stringifier. All HTML transformations are made by plugins. And these plugins are just small plain JS functions, which receive a HTML node tree, transform it, and return a modified tree. For more detailed information about PostHTML in general take a look at the docs.	Dependencies <table><tr><th>Name</th><th>Status</th><th>Description</th></tr><tr><td>posthtml-parser</td><td></td><td>Parser HTML/XML to PostHTMLTree</td></tr></table>	Name	Status	Description	posthtml-parser		Parser HTML/XML to PostHTMLTree	<table><tr><th>Name</th><th>Status</th><th>Description</th></tr><tr><td>posthtml-render</td><td></td><td>Render PostHTMLTree to HTML/XML</td></tr></table>	Name	Status	Description	posthtml-render		Render PostHTMLTree to HTML/XML	<table><tr><th>Name</th><th>Status</th><th>Description</th></tr><tr><td colspan="3">Create to your project</td></tr><tr><td colspan="3">npm init posthtml</td></tr><tr><td colspan="3">Install</td></tr><tr><td colspan="3">npm i -D posthtml</td></tr></table>	Name	Status	Description	Create to your project			npm init posthtml			Install			npm i -D posthtml		
Name	Status	Description																												
posthtml-parser		Parser HTML/XML to PostHTMLTree																												
Name	Status	Description																												
posthtml-render		Render PostHTMLTree to HTML/XML																												
Name	Status	Description																												
Create to your project																														
npm init posthtml																														
Install																														
npm i -D posthtml																														
9 16 <pre>import posthtml from 'posthtml' const html = ` <html>   <body>     <p class="wow">OMG</p>   </body> </html> `, posthtml([</pre>	10 15 Async Super Title Awsome Text Warning: Async Plugins can't be used in sync mode and will throw an Error. It's recommended to use PostHTML asynchronously whenever possible.	11 14 <pre>const result = posthtml() .use(require('posthtml-custom-elements'))() .process(html, { sync: true }) .html console.log(result)</pre>	12 13 API Sync <pre>import posthtml from 'posthtml' const html = ` <component> <title>Super Title</title> <text>Awesome Text</text></pre> Usage																											

```
require('posthtml-to-svg-tags')(),
require('posthtml-extend-attrs')({
  attrTree: {
    'wow': {
      id: 'wow_id',
      fill: '#4A83B4',
      'fill-rule': 'evenodd',
      'font-family': 'Verdana'
    }
  }
})
})
```

17

```
.process(html/*, options */)
.then((result) =>
  console.log(result.html))

<svg xmlns="http://www.w3.org/2000/
  svg">

  <text
    class="wow"
    id="wow_id"
    fill="#4A83B4"
    fill-rule="evenodd" font-
      family="Verdana">
```

18

```

</text>
</svg>

```

Directives

```

import posthtml from 'posthtml'

const php = `
  <component>
    <title><?php echo $title; ?></
      title>
    <text><?php echo $article; ?></
      text>

```

19

```

    </component>
  },
  const result = posthtml()
    .use(require('posthtml-custom-
      elements'))
    .process(html, {
      directives: [
        { name: '?php', start: '<',
          end: '>' }
      ]
    })
  })
}

```

20

```
src('src/**/*.html')
    .pipe(tap((file) => path =
        file.path))
    .pipe(posthtml({plugins, options}))
    .pipe(dest('build/'))
})
```

24

```
import tap from 'gulp-tap',
import posthtml from 'gulp-posthtml',
import { task, src, dest } from 'gulp',
let path
task('html', () => {
  const plugins = [ require('posthtml-include'), root: '${path}' ]
  const options = { }
```

23

```

"scripts": {
  "build": "webpack --mode=development --devtool=cheap-module-source-map"
},
"devDependencies": {
  "webpack": "^4.41.2",
  "webpack-cli": "^3.3.10",
  "webpack-dev-server": "^3.11.0",
  "webpack-merge": "^5.7.3"
},
"scripts": {
  "start": "webpack serve --mode=development --devtool=cheap-module-source-map"
}

```

22

```

<html>
<script>
    console.log(result)
</script>
</html>

```

21

```
posthtml: {
  options: {
    use: [
      require('posthtml-doctype')({
        doctype: 'HTML 5' }),
      require('posthtml-include')({
        root: './', encoding:
          'utf-8' })
    ]
  },
  build: {
    files: [
```

25

```
{
  dot: true,
  cwd: 'html/',
  src: ['*.html'],
  dest: 'tmp/',
  expand: true,
}
```

26

```
Webpack
```

```
npm i -D html-loader posthtml-loader
```

```
v1.x
```

```
webpack.config.js
```

```
const config = {  
  module: {  
    loaders: [  
      {  

```

27

```

    test: /\.html$/,
    loader: 'html!posthtml'
  }
]
},
posthtml: (ctx) => ({
  parser: require('posthtml-pug'),
  plugins: [
    require('posthtml-bem')()
  ]
})
}

```

28

```
(require('posthtml-beam'
    [
      {
        }
      }
    ]
  )
```

32

```

    },
    plugins: [
        new LoaderOptionsPlugin({
            options: {
                posthtml(ctx) {
                    return {
                        parser: require('posthtml-
31

```

31

```
test: /\.html$/,
  use: [
    {
      loader: 'html-loader',
      options: { minimize: true },
      loader: 'posthtml-loader'
    }
  ]
}
```

30

```
export default config

v2.x

webpack.config.js

import { LoaderOptionsPlugin } from 'webpack'

const config = {
  rules: [

```

67

Rollup

```
$ npm i rollup-plugin-posthtml -D
# or
$ npm i rollup-plugin-posthtml-
  template -D

import { join } from 'path';

import posthtml from 'rollup-plugin-
  posthtml-template';

// or
```

33

```
// import posthtml from 'rollup-
  plugin-posthtml';

import sugarml from 'posthtml-
  sugarml'; // npm i posthtml-
  sugarml -D

import include from 'posthtml-
  include'; // npm i posthtml-
  include -D

export default {
  entry: join(__dirname, 'main.js'),
```

34

```
dest: join(__dirname, 'bundle.js'),
format: 'iife',
plugins: [
  posthtml({
    parser: sugarml(),
    plugins: [include()],
    template: true // only rollup-
                  plugin-posthtml-template
  })
];
```

35

Parser

```
import pug from 'posthtml-pug'

posthtml().process(html, { parser:
  pug(options) }).then((result)
=> result.html)
```

Name	Status	Description
John Doe	Active	Software Engineer
Jane Smith	Inactive	Product Manager
Bob Johnson	Active	Marketing Specialist
Alice Brown	Active	UX Designer
Charlie Davis	Active	Frontend Developer
Eve Wilson	Active	Backend Developer
Frank Miller	Active	QA Tester
Grace Lee	Active	Operations Manager
Henry Taylor	Active	Business Development
Ivy White	Active	Customer Support
Jack Black	Active	Systems Administrator
Karen Green	Active	HR Manager
Leo Clark	Active	Finance Analyst
Mia Hall	Active	Legal Counsel
Noah King	Active	IT Support
Olivia Scott	Active	Project Manager
Peter Young	Active	Sales Representative
Quinn Adams	Active	Business Development
Rachel Baker	Active	Marketing Specialist
Samuel Carter	Active	UX Designer
Tina Evans	Active	Frontend Developer
Victor Foster	Active	Backend Developer
Wendy Gibson	Active	QA Tester
Xavier Harris	Active	Operations Manager
Yara Ives	Active	Business Development
Zoe King	Active	Customer Support

[posthtml-pug](#) Pug Parser

36



40

Contributors

```

Ivan Demidov
height="150" src="https://avatars.githubusercontent.com/u/1510217?v=3&s=150">
Ivan Voïschey
```

68

Maintainers

- In case you want to develop your own plugin, use the recommended `posthtml-plugin-starter`
- `posthtml-plugin-starter` is a recommended plugin to get started.
- `awesome-posthtml` is a recommended plugin to get started.

38

Name	Status	Description
Plugins		

SugarML Parser

ਸੁਗਰਮ

Status Description

37

Backers

Thank you to all our backers! 🙏 [[Become a backer](#)]



41

42

43

44

48

47

46

45